

Strings

In Java, a String is an object that represents a sequence of characters.

Akash $\Rightarrow$ String

↓
A K A S H $\Rightarrow$ sequence of char

Package $\Rightarrow$ java.lang

Strings are immutable $\Rightarrow$ once a object is created, its value cannot be changed.

String $\rightarrow$ class $\rightarrow$ Object $\rightarrow$ Heap memory

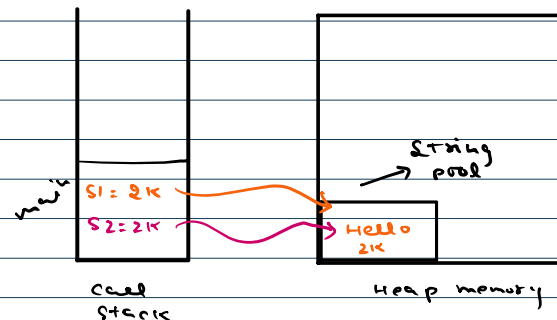
↓
Address will be stored
in variable inside
call stack.

String pool

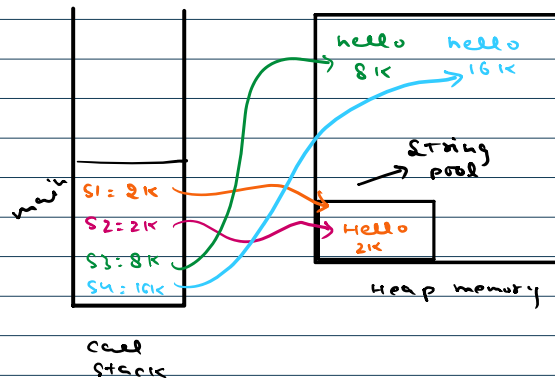
```
String s1 = "Akash";
String s2 = new String("Akash");
```

$\rightarrow$ Heap memory

```
public class StringDemo {
    public static void main(String args[]) {
        String s1 = "hello";
        String s2 = "hello";
    }
}
```



```
public class StringDemo {
    public static void main(String args[]) {
        String s1 = "hello";
        String s2 = "hello";
        String s3 = new String(original: "hello");
        String s4 = new String(original: "hello");
    }
}
```



String concatenation

+ $\Rightarrow$ to combine two or more than two strings.

what happens when we modify the string?

```
public class StringPoolDemo {
    public static void main(String args[]) {
        String s1 = "hello";
        String s2 = "hello";

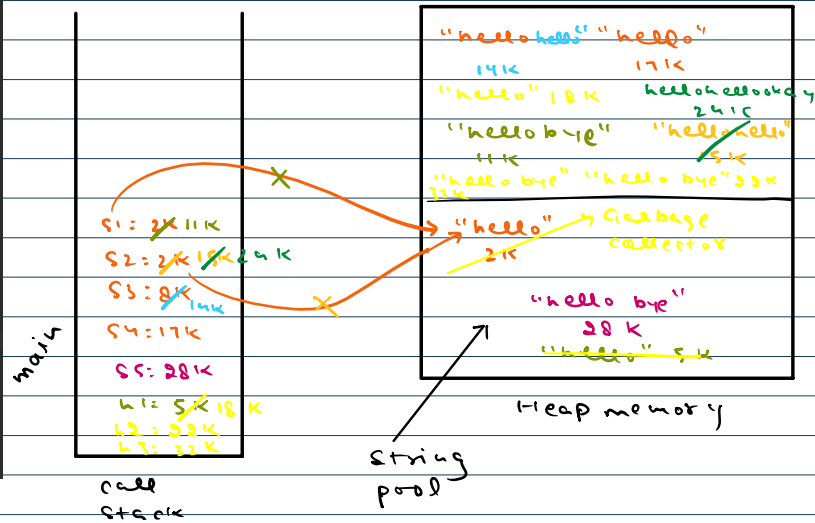
        String s3 = new String(original: "hello");
        String s4 = new String(original: "hello");

        s1 = s1 + "bye";
        s2 = s2 + s3;
        s3 = s3 + s4;

        s2 = s2.concat(" bye");
        String s5 = "hello" + "bye";

        String h1 = "hello";

        String h2 = h1 + new String(original: "bye");
        String h3 = h1 + new String(original: "bye");
        System.out.println(h2 == h3);
    }
}
```



String + [any datatype] = String

charAt

String s = "Akash"

s[0] = 'a'

s.charAt(0) = 'a'

compareTo() → comparing strings lexicographically.

s1.compareTo(s2);

0 → both strings are equal.

<0 → if s1 comes before s2

>0 → if s2 comes before s1

s1 = "Abhay"
s2 = "Akash"

What is a substring?

It is a contiguous sequence of chars within a string.

inord → "akash"

"ak", "ka", "ash", "sh", "s", "h"

substring() ⇒ to extract a portion of a string.

s.substring(startIndex, endIndex);

inclusive

exclusive
{ optional }

Print all substrings

~~~~~

0 1 2 3 4 5  
 a k a r s h  
 +1  
 0-1 a  
 0-2 a k  
 0-3 a k a  
 0-4 a k a r  
 0-5 a k a r s  
 0-6 a k a r s h

+1  
 1-2 k  
 1-3 k a  
 1-4 k a r  
 1-5 k a r s  
 1-6 k a r s h

+1  
 2-3 a  
 2-4 a r  
 2-5 a r s  
 2-6 a r s h

+1  
 3-4 r  
 3-5 r s  
 3-6 r s h

+1  
 4-5 s  
 4-6 s h

+1  
 5-6 h

```
for (int i=0; i<s.length(); i++) {
    for (int j=i+1; j<=s.length(); j++) {
        s.substring(i, j);
    }
}
```

<https://leetcode.com/problems/reverse-words-in-a-string/>